

OpenMP: diretivas, clausulas e politicas mais usadas

Este arquivo resume as principais construcoes do OpenMP em C/C++.

Observacao importante: alguns termos muito usados em OpenMP **nao sao diretivas isoladas**. Por exemplo:

- `num_threads` e uma clausula usada em `parallel`
- `reduction` e uma clausula
- `static` normalmente e um tipo de escalonamento em `schedule(static)`

Mesmo assim, todos esses itens estao explicados aqui porque aparecem com frequencia em exercicios e codigos paralelos.

1. Diretivas de paralelismo basico

`#pragma omp parallel`

Cria uma regio paralela. O bloco seguinte passa a ser executado por varias threads.

```
#pragma omp parallel
{
    printf("Thread %d de %d\n", omp_get_thread_num(), omp_get_num_threads());
}
```

Quando usar:

- para dividir um trecho independente entre varias threads

Cuidados:

- variaveis compartilhadas podem causar condicao de corrida

`#pragma omp for`

Divide as iteracoes de um laço `for` entre as threads de uma regio paralela ja existente.

```
#pragma omp parallel
{
    #pragma omp for
    for (int i = 0; i < n; i++) {
        a[i] = b[i] + c[i];
    }
}
```

Quando usar:

- para paralelizar lacos com iteracoes independentes

Cuidados:

- so funciona corretamente quando nao ha dependencia entre iteracoes

`#pragma omp parallel for`

Atalho para combinar `parallel` com `for`.

```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    a[i] = b[i] + c[i];
}
```

Quando usar:

- quando a unica finalidade da regioao paralela e executar o laco

`#pragma omp sections`

Divide blocos distintos de codigo entre as threads.

```
#pragma omp parallel sections
{
    #pragma omp section
    tarefa1();

    #pragma omp section
    tarefa2();
}
```

Quando usar:

- quando existem tarefas diferentes e independentes

`#pragma omp section`

Define cada bloco individual dentro de `sections`.

`#pragma omp single`

Garante que apenas uma thread execute o bloco.

```
#pragma omp parallel
{
    #pragma omp single
    inicializar_dados();
}
```

Quando usar:

- leitura de arquivo
- inicializacao
- impressao centralizada

#pragma omp master

Faz o bloco ser executado apenas pela thread mestre.

```
#pragma omp parallel
{
    #pragma omp master
    printf("Executado pela thread mestre\n");
}
```

Observacao:

- em versoes mais novas do OpenMP, `masked` e a forma mais moderna e flexivel

#pragma omp masked

Semelhante a `master`, mas mais geral. Permite controlar qual thread executa o bloco.

#pragma omp barrier

Cria uma barreira de sincronizacao. Todas as threads precisam chegar nesse ponto antes de continuar.

```
#pragma omp parallel
{
    etapa1();

    #pragma omp barrier

    etapa2();
}
```

Quando usar:

- quando uma fase depende da conclusao de outra

#pragma omp ordered

Mantem uma parte do laco em ordem sequencial, mesmo dentro de um laco paralelo.

```
#pragma omp parallel for ordered
for (int i = 0; i < n; i++) {
    calcular(i);
}
```

```
#pragma omp ordered
salvar_em_ordem(i);
}
```

Quando usar:

- quando parte do processamento precisa respeitar a ordem do laço

2. Sincronizacao e protecao de dados

#pragma omp critical

Permite que apenas uma thread por vez entre em um bloco critico.

```
#pragma omp critical
{
    soma_global += valor_local;
}
```

Quando usar:

- atualizacao de recurso compartilhado
- impressao sem misturar saidas

Cuidados:

- pode reduzir bastante o desempenho se usado em excesso

#pragma omp atomic

Protege uma operacao simples de leitura-modificacao-escrita.

```
#pragma omp atomic
soma++;
```

Diferenca para `critical`:

- `atomic` e mais leve
- funciona apenas para operacoes simples

#pragma omp flush

Forca sincronizacao de memoria entre threads.

Quando usar:

- casos mais avancados de consistencia de memoria

Observacao:

- em codigos basicos, `barrier`, `atomic` e `critical` costumam ser suficientes

3. Diretivas de tarefas

`#pragma omp task`

Cria uma tarefa que pode ser executada depois por alguma thread disponivel.

```
#pragma omp parallel
{
    #pragma omp single
    {
        #pragma omp task
        processar_bloco(1);

        #pragma omp task
        processar_bloco(2);
    }
}
```

Quando usar:

- recursao
- arvores
- grafos
- cargas de trabalho irregulares

`#pragma omp taskwait`

Faz a thread esperar a conclusao das tarefas filhas ja criadas.

```
#pragma omp taskwait
```

`#pragma omp taskgroup`

Agrupa tarefas para sincronizacao conjunta.

`#pragma omp taskloop`

Cria tarefas a partir de um laco.

Quando usar:

- quando voce quer o modelo de tarefas, nao apenas divisao fixa de iteracoes

4. Vetorizacao e paralelismo hibrido

`#pragma omp simd`

Pede vetorizacao do laco usando instrucoes SIMD.

```
#pragma omp simd
for (int i = 0; i < n; i++) {
    a[i] = b[i] * c[i];
}
```

Quando usar:

- calculos numericos em vetores e matrizes

`#pragma omp parallel for simd`

Combina threads com vetorizacao.

`#pragma omp declare simd`

Indica que uma funcao pode ter versao vetorizada.

5. Offloading e aceleradores

Essas diretivas sao usadas quando o programa envia trabalho para GPU ou outro acelerador.

`#pragma omp target`

Move execucao para um dispositivo alvo.

`#pragma omp target data`

Controla regio de dados no dispositivo.

`#pragma omp target update`

Atualiza dados entre host e dispositivo.

`#pragma omp teams`

Cria grupos de threads no dispositivo.

`#pragma omp distribute`

Distribui iteracoes entre equipes.

`#pragma omp target teams distribute parallel for`

Combinacao comum para GPU.

Observacao:

- em disciplinas introdutorias, essas diretivas podem nao aparecer tanto quanto `parallel for`, `critical` e `reduction`

6. Clausulas mais importantes

As clausulas alteram o comportamento das diretivas.

`num_threads(n)`

Define quantas threads a regioao paralela deve tentar usar.

```
#pragma omp parallel num_threads(8)
```

Tambem pode ser controlado pela variavel de ambiente `OMP_NUM_THREADS`.

`private(var)`

Cada thread recebe sua propria copia da variavel.

`firstprivate(var)`

Cada thread recebe uma copia inicializada com o valor original.

`lastprivate(var)`

Ao final do laco/bloco, copia o valor da ultima iteracao logica para fora.

`shared(var)`

Define que a variavel sera compartilhada entre as threads.

`default(shared) / default(none)`

Controla a regra padrao para escopo das variaveis.

Boa pratica:

- `default(none)` forca declarar explicitamente o escopo e evita erros

`reduction(op:var)`

Cada thread acumula um valor local e o OpenMP combina tudo ao final.

```
int soma = 0;

#pragma omp parallel for reduction(+:soma)
for (int i = 0; i < n; i++) {
    soma += v[i];
}
```

Quando usar:

- soma
- produto
- minimo
- maximo
- contadores

Vantagem:

- normalmente e melhor que usar `critical` dentro do laço

`schedule(tipo)`

Define como as iteracoes do laço serao distribuidas.

Tipos principais:

`schedule(static)`

Divide as iteracoes de forma fixa entre as threads.

Quando usar:

- carga balanceada
- baixo overhead

Exemplo:

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < n; i++) {
    trabalho(i);
}
```

`schedule(dynamic)`

As threads recebem novos blocos de iteracoes conforme terminam os anteriores.

Quando usar:

- carga irregular

Vantagem:

- melhora balanceamento

Desvantagem:

- maior overhead

`schedule(guided)`

Semelhante a `dynamic`, mas começa com blocos maiores e depois reduz.

Quando usar:

- carga irregular com tentativa de reduzir overhead

`schedule(auto)`

Deixa a escolha para compilador/runtime.

`schedule(runtime)`

Usa configuracao definida em variavel de ambiente, como `OMP_SCHEDULE`.

`collapse(n)`

Colapsa `n` laços aninhados em um unico espaco de iteracao.

```
#pragma omp parallel for collapse(2)
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        a[i][j] = i + j;
    }
}
```

Quando usar:

- matrizes
- laços aninhados com pouca carga por iteracao

`nowait`

Remove a barreira implicita no final de algumas diretivas como `for` e `sections`.

Quando usar:

- quando nao e necessario esperar todas as threads

`if(condicao)`

Permite ativar ou nao uma regio paralela dependendo de uma condicao.

```
#pragma omp parallel for if(n > 1000)
for (int i = 0; i < n; i++) {
    trabalho(i);
}
```

Quando usar:

- para evitar overhead em problemas pequenos

ordered

Clausula usada junto com lacos para permitir bloco `ordered`.

copyin

Copia para as threads valores de variaveis `threadprivate`.

copyprivate

Usada com `single` para distribuir valores produzidos por uma thread as demais.

7. Diretivas auxiliares e declaracoes

#pragma omp threadprivate

Torna uma variavel global/estatica privada por thread.

#pragma omp declare reduction

Permite criar reducoes personalizadas.

#pragma omp declare target

Marca funcoes ou dados que podem ser usados no dispositivo alvo.

#pragma omp requires

Declara requisitos globais do programa para OpenMP.

8. Regras praticas para escolher a construcao correta

Use `parallel for` quando:

- o problema for um laço com iteracoes independentes

Use `reduction` quando:

- varias threads precisam acumular um resultado comum

Use `critical` quando:

- o bloco compartilhado e pequeno, mas nao cabe em `atomic`

Use `atomic` quando:

- a operacao for simples, como `++`, `+=`, `--`

Use `schedule(static)` quando:

- todas as iteracoes tem custo parecido

Use `schedule(dynamic)` quando:

- algumas iteracoes sao muito mais pesadas que outras

Use `sections` quando:

- existem poucas tarefas bem diferentes entre si

Use `task` quando:

- a carga e irregular ou criada dinamicamente

9. Exemplo completo

```
#include <omp.h>
#include <stdio.h>

int main() {
    int soma = 0;

    #pragma omp parallel for num_threads(4) reduction(+:soma) schedule(static)
    for (int i = 1; i <= 100; i++) {
        soma += i;
    }

    printf("Soma = %d\n", soma);
    return 0;
}
```

Esse exemplo usa:

- `parallel for`: paraleliza o laço
- `num_threads(4)`: pede 4 threads
- `reduction(+:soma)`: evita condicao de corrida na soma
- `schedule(static)`: distribui iteracoes de forma fixa

10. Resumo rapido

As construcoes mais comuns em atividades de OpenMP costumam ser:

- `parallel`
- `for`
- `parallel for`
- `critical`
- `atomic`
- `barrier`
- `sections`
- `single`
- `task`
- `reduction`
- `schedule(static|dynamic|guided)`

- `private`, `shared`, `firstprivate`
- `collapse`
- `nowait`

Se quiser, este material pode ser estendido depois com:

- exemplos em C para cada diretiva
- comparacao entre `critical`, `atomic` e `reduction`
- tabela de desempenho e casos de uso